

EE 446 TinyML Lab 8 Report

Part I Edge Impulse project: <https://studio.edgeimpulse.com/public/1013948/live>

Part I: Edge Impulse Output

The Edge Impulse live classification project was run using the public project link above. The output excerpt shows repeated inference cycles with the expected gesture labels and anomaly score.

```
Sampling...
Predictions (DSP: 109 ms., Classification: 0 ms., Anomaly: 1 ms.):
  idle: 0.004891
  snake: 0.987623
  updown: 0.000000
  wave: 0.003486
  anomaly score: -0.214672
Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 109 ms., Classification: 0 ms., Anomaly: 1 ms.):
  idle: 0.000000
  snake: 0.995342
  updown: 0.000000
  wave: 0.004658
  anomaly score: -0.069418
Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 109 ms., Classification: 0 ms., Anomaly: 1 ms.):
  idle: 0.995823
  snake: 0.000000
  updown: 0.000000
  wave: 0.000000
  anomaly score: -0.207843
Starting inferencing in 2 seconds...
```

Figure 1. Edge Impulse serial output excerpt.

Part II: Tiny Ensemble Learning

Question 1: Model Architecture and Ensemble Flow

Each sample is a flattened IMU window with 100 time steps and 6 sensor features, so the model input dimension is 600. The ensemble uses three parallel branches that receive raw, standard-scaled, and min-max-scaled versions of the same window.

TinyML Lab 8 Three-Branch Stacked Ensemble

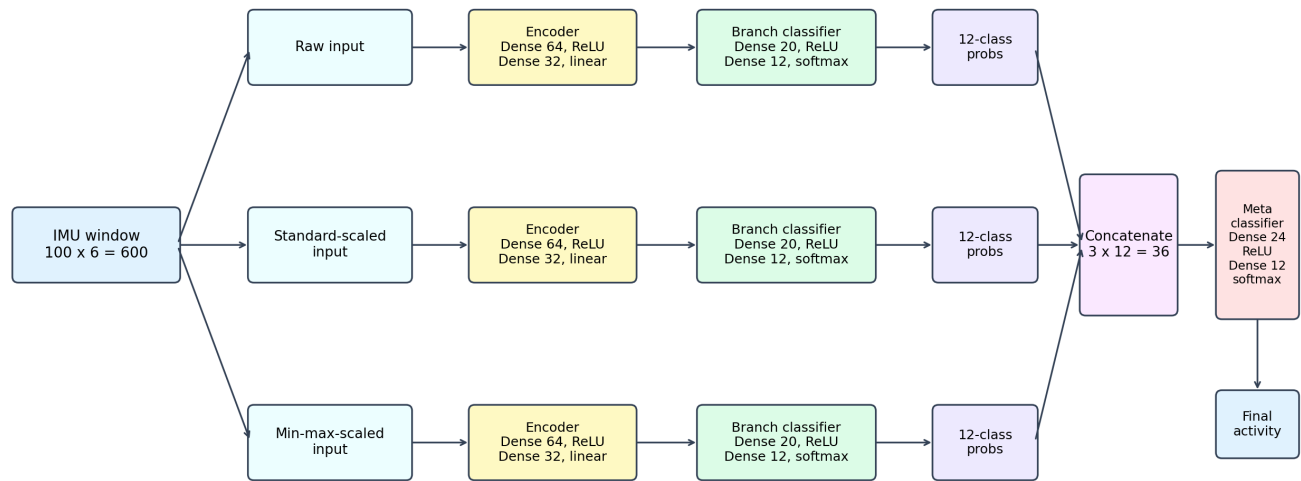


Figure 2. Three-branch stacked ensemble architecture.

- Encoder in each branch: 600 -> Dense(64, ReLU) -> Dense(32, linear).
- Branch classifier: 32 -> Dense(20, ReLU) -> Dense(12, softmax).
- Each branch outputs 12 class probabilities; the three outputs are concatenated into 36 inputs for the meta-classifier.
- Stacked meta-classifier: 36 -> Dense(24, ReLU) -> Dense(12, softmax).
- The deployed ensemble contains four models: three branch models and one meta-classifier.

Question 2: Pruning and QAT Methodology

The deployment models were compressed with pruning followed by quantization-aware training. Pruning uses TensorFlow Model Optimization's low-magnitude pruning with a polynomial sparsity schedule from 10% initial sparsity to 80% final sparsity, with `begin_step = 0` and `end_step = 500`. The local run reached about 79.44% sparsity for each branch model and 79.43% for the stacked meta-classifier.

After pruning, the pruning wrappers are stripped so QAT can wrap standard Keras Dense layers. The stripped model keeps the sparse weights, then fake-quantization nodes are inserted and the model is fine-tuned. A mask-enforcement callback reapplies the pruning masks after each batch and epoch so previously pruned weights remain zero during QAT.

The final TFLite export uses full integer quantization with int8 input and output tensors. Representative data is needed so the converter can calibrate activation ranges and choose usable int8 scale and zero-point values.

Model	int8 size	TFLite accuracy	Sparsity
encoder_clf_raw	44.55 KB	0.9212	79.44%
encoder_clf_std	44.55 KB	0.9912	79.44%
encoder_clf_minmax	44.61 KB	0.8546	79.44%
stacked_meta_clf	3.63 KB	0.9939	79.43%

Table 1. Local compressed-model run results.

Question 3: Arduino Deployment and Real-World Behavior

```
19:17:57.123 -> Samples collected: 25
19:17:57.681 -> Samples collected: 50
19:17:58.208 -> Samples collected: 75
19:17:58.765 -> Samples collected: 100
19:17:58.765 -> Window collected. Running ensemble inference...
19:17:58.765 -> Raw model scores: 0.0000, 0.0000, 0.9948, 0.0000, 0.0000, 0.0000, 0.0041,
    0.0000, 0.0000, 0.0000, 0.0000, 0.0000
19:17:58.765 -> Standard-scaled model scores: 0.0000, 0.9948, 0.0000, 0.0000, 0.0000, 0.0000,
    0.0041, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
19:17:58.798 -> Min-max-scaled model scores: 0.0000, 0.0000, 0.3512, 0.0000, 0.5552, 0.0000,
    0.0109, 0.0814, 0.0000, 0.0000, 0.0000, 0.0000
19:17:58.798 -> Meta-classifier scores: 0.0194, 0.1647, 0.6086, 0.0626, 0.0000, 0.0042, 0.0000,
    0.0392, 0.0663, 0.0274, 0.0000, 0.0076
19:17:58.798 -> =====
19:17:58.798 -> Final prediction: Lying down
19:17:58.798 -> Class index: 2
19:17:58.798 -> Confidence score: 0.6086
19:17:58.798 -> =====
19:17:59.324 -> Samples collected: 25
19:17:59.883 -> Samples collected: 50
19:18:00.413 -> Samples collected: 75
19:18:00.937 -> Samples collected: 100
19:18:00.937 -> Window collected. Running ensemble inference...
19:18:00.968 -> Raw model scores: 0.0000, 0.0000, 0.9945, 0.0000, 0.0000, 0.0000, 0.0042,
    0.0000, 0.0000, 0.0000, 0.0000, 0.0000
19:18:00.968 -> Standard-scaled model scores: 0.0000, 0.9945, 0.0000, 0.0000, 0.0000, 0.0000,
    0.0042, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
19:18:00.968 -> Min-max-scaled model scores: 0.0000, 0.0000, 0.3523, 0.0000, 0.5539, 0.0000,
    0.0110, 0.0831, 0.0000, 0.0000, 0.0000, 0.0000
19:18:01.001 -> Meta-classifier scores: 0.0196, 0.1640, 0.6079, 0.0625, 0.0000, 0.0041, 0.0000,
    0.0391, 0.0665, 0.0273, 0.0000, 0.0077
19:18:01.001 -> Final prediction: Lying down
19:18:01.001 -> Class index: 2
19:18:01.001 -> Confidence score: 0.6079
```

Figure 3. Arduino serial monitor output excerpt for the resting board test.

For a board lying flat on a stable surface, the dominant signal is gravity on one accelerometer axis with very small gyro motion. A static prediction such as Lying down, Standing still, or Sitting and relaxing is acceptable because those classes are closest to a non-moving board. The shown run predicts Lying down with meta-classifier confidence around 0.60, which is reasonable for a resting-board condition.

Observed issues include mismatch between body-worn mHealth sensors and a desk-resting Arduino, axis-orientation differences, possible unit conversion differences, and small sensor noise. Two practical improvements are to verify the acceleration and gyroscope unit conversion before scaling and to collect a small calibration dataset using the actual Arduino orientation.

Question 4: Deployment Behavior on the Arduino

Unexpected behavior may include repeatedly predicting one class or cycling among only a few labels when the board is moved differently. This can happen because the model was trained on human body-mounted mHealth data, while the live Arduino board is hand-held or desk-resting. Gravity, sensor placement, motion frequency, and axis orientation therefore do not match the training distribution.

The compression step can make this more visible: with about 80% sparsity and int8 precision, ambiguous out-of-distribution inputs may collapse toward whichever class has the largest sparse integer response. Better deployment reliability would require collecting Arduino-specific examples or recalibrating the scaling pipeline with live board data.